

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



PRACTICA Nº 08

NOMBRE COMPLETO: COLIMA ESQUIVEL YADIRA

Nº de Cuenta: 422115533

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE: 2027-1

FECHA DE ENTREGA LÍMITE: 21/10/2026

CALIFICACIÓN: _____

Ejercicios solicitados

Ejercicio 1:

1.- Importar un modelo de una nave espacial bien texturizada que tenga dos motores o reactores y añadirle 1 luz puntual y 1 luz de tipo spotlight. Acomodar a la nave flotando



Figura 1

Ejercicio 2:

2.- Con una tecla prender y apagar al mismo tiempo las luces añadidas a la nave espacial



Figura 2

Con la letra G se apagan las luces azul spotlight y morada luz puntual.
La nave se mueve con L J K I

Ejercicio 3:

3.- Añadirle a cada uno de los motores una textura que simule fuego o plasma de color azul saliendo de él y añadir en la posición de cada textura de fuego una luz de tipo puntual de color azul.
4.- Con dos teclas independientes añadir que al presionar la tecla se prenda la luz y se vea la textura del fuego de cada motor, de tal forma que se logre ver 4 estados: los dos motores apagados, los dos motores prendidos, motor 1 apagado y motor 2 prendido, motor 1 prendido y motor 2 apagado.

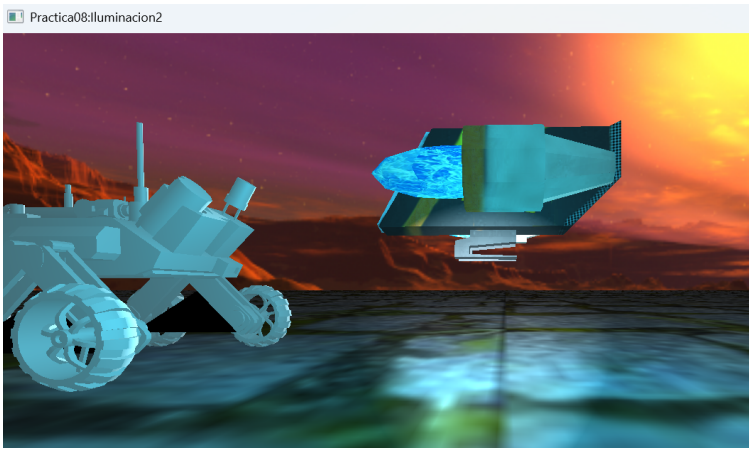


Figura 3



Figura 4



Figura 5

Con la tecla 1 luz derecha y 2 luz izquierda

Bloque de código

Función principal ciclo while

```
1
2         while (!mainWindow.getShouldClose())
3         {
4                 GLfloat now = glfwGetTime();
5                 deltaTime = now - lastTime;
6                 deltaTime += (now - lastTime) / limitFPS;
7                 lastTime = now;
8
9                 //Recibir eventos del usuario
10                glfwPollEvents();
11                camera.keyControl(mainWindow.getsKeys(), deltaTime);
12                camera.mouseControl(mainWindow.getXChange(), mainWindow.
                     getYChange());
13
14                // Clear the window
15                glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
16                glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
17                skybox.DrawSkybox(camera.calculateViewMatrix(), projection);
18                shaderList[0].UseShader();
19                uniformModel = shaderList[0].GetModelLocation();
20                uniformProjection = shaderList[0].GetProjectionLocation();
21                uniformView = shaderList[0].GetViewLocation();
22                uniformEyePosition = shaderList[0].GetEyePositionLocation();
23                uniformColor = shaderList[0].getColorLocation();
24
25                //informaci n en el shader de intensidad especular y brillo
26                uniformSpecularIntensity = shaderList[0].
                     GetSpecularIntensityLocation();
27                uniformShininess = shaderList[0].GetShininessLocation();
28
29                glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr
                     (projection));
30                glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(
                     camera.calculateViewMatrix()));
31                glUniform3f(uniformEyePosition, camera.getCameraPosition().x,
                     camera.getCameraPosition().y, camera.getCameraPosition().z);
32
33                // luz ligada a la c mara de tipo flash
34                //sirve para que en tiempo de ejecuci n (dentro del while) se
                     cambien propiedades de la luz
35                lowerLight = camera.getCameraPosition();
36                lowerLight.y -= 0.3f;
37                spotLights[0].SetFlash(lowerLight, camera.getCameraDirection());
38
39
40                // Se actualiza cada frame con la tecla
41                roverPosicionA.x = mainWindow.getRoverX();
42                //ara nave
43                navePosicionA.x = mainWindow.getAvionX();
44                navePosicionA.y = mainWindow.getAvionY();
45                //para
46
```

```
47                  //matriz
48                  glm::mat4 roverBase = glm::translate(glm::mat4(1.0f),
                       roverPosicionA);
49                  glm::mat4 naveBase = glm::translate(glm::mat4(1.0f),
                       navePosicionA);
50
51                  glm::mat4 naveFuegoDerBase = glm::translate(naveBase,
                       naveFuegoDerOffset);
52                  glm::mat4 naveFuegoIzqBase = glm::translate(naveBase,
                       naveFuegoIzqOffset);
53
54
55
56                  //
57                  // Posici n del faro del rover
58                  glm::vec3 faroPosRov = glm::vec3(roverBase * glm::vec4(
                       faroRoverOffset, 1.0f));
59
60                  // Direcci n del faro hacia adelante X
61                  glm::vec3 faroDirRov = glm::normalize(glm::mat3(roverBase) *
                       faroRoverDir);
62
63                  // Actualizar el spotlight del faro cada frame
64                  spotLights[2].SetFlash(faroPosRov, faroDirRov);
65
66                  //
67
68                  //
69                  // Posici n del faro del avion
70                  glm::vec3 faroPosA = glm::vec3(naveBase * glm::vec4(
                       faroNaveIzqOffset, 1.0f));
71
72                  // Direcci n del faro hacia adelante X
73                  glm::vec3 faroDirA = glm::normalize(glm::mat3(naveBase) *
                       faroNaveIzqDir);
74
75                  // Actualizar el spotlight del faro cada frame
76                  spotLights[3].SetFlash(faroPosA, faroDirA);
77
78                  // LUZ PUNTUAL NAVE mover con la nae
79                  // Posici n del faro de nave luz puantual
80                  glm::vec3 faroPosA2 = glm::vec3(naveBase * glm::vec4(
                       faroNaveDerOffset, 1.0f));
81
82                  // Actualizar el poinLight del faro cada frame
83                                  // Actualizar el pointLight de la nave cada frame
                                        (una luz puntual solo tiene posicion, no
                                       direccion)
84                  pointLights[2] = PointLight(0.5f, 0.0f, 0.95f,//luz morada
85                          1.0f, 0.8f,
86                          faroPosA2.x, faroPosA2.y, faroPosA2.z,
87                          1.0f, 0.6f, 0.01f);
```

```
88
89              // LUCES PUNTUALES AZULES de los fuegos, en la posicion de cada
                   llama (se mueven con la nave)
90              glm::vec3 luzFuegoIzqPos = glm::vec3(naveFuegoIzqBase * glm::vec4
                   (0.0f, 0.0f, 0.0f, 1.0f)) + luzFuegoAjuste;
91              glm::vec3 luzFuegoDerPos = glm::vec3(naveFuegoDerBase * glm::vec4
                   (0.0f, 0.0f, 0.0f, 1.0f)) + luzFuegoAjuste;
92              pointLights[3] = PointLight(0.0f, 0.831f, 1.0f,//luz azul fuego
                   izq
93                      1.0f, 0.8f,
94                      luzFuegoIzqPos.x, luzFuegoIzqPos.y, luzFuegoIzqPos.z,
95                      1.0f, 0.3f, 0.1f);
96              pointLights[4] = PointLight(0.0f, 0.821f, 0.95f,//luz azul fuego
                   der
97                      1.0f, 0.8f,
98                      luzFuegoDerPos.x, luzFuegoDerPos.y, luzFuegoDerPos.z,
99                      1.0f, 0.3f, 0.1f);
100
101
102             ////holocron
103             //// Posici n del faro del avion
104             //glm::vec3 faroPosA = glm::vec3(avionBase * glm::vec4(
                   faroAvionOffset, 1.0f));
105             //glm::vec3 faroDirA = glm::normalize(glm::mat3(avionBase) *
                   faroAvionDir);
106             //spotLights[5].SetFlash(-7,5,0, faroDirA);
107
108
109
110             //informaci n al shader de fuentes de iluminaci n
111             shaderList[0].SetDirectionalLight(&mainLight);
112             // las luces de la nave: el spotlight es el ultimo de los
                   spotlights, apagado se manda un conteo menos
113             if (mainWindow.getLucesNave())
114             {
115                     shaderList[0].SetSpotLights(spotLights, spotLightCount);
116             }
117             else
118             {
119                     shaderList[0].SetSpotLights(spotLights, spotLightCount -
                           1);
120             }
121
122             // puntuales: apagadas se arma un arreglo sin la morada de la
                   nave ([2]), cada motor apaga su luz azul con su propia tecla
                   (1 y 2)
123             PointLight pointLightsMostrar[MAX_POINT_LIGHTS];
124             unsigned int pointLightCountMostrar = 0;
125             for (unsigned int i = 0; i < pointLightCount; i++)
126             {
127                     if (i == 2 && !mainWindow.getLucesNave())
128                     {
```

```
129                     continue;
130             }
131             if (i == 3 && !mainWindow.getMotor1()) // luz azul del
                    motor 1 (fuego izq)
132             {
133                     continue;
134             }
135             if (i == 4 && !mainWindow.getMotor2()) // luz azul del
                    motor 2 (fuego der)
136             {
137                     continue;
138             }
139             pointLightsMostrar[pointLightCountMostrar] = pointLights[
                    i];
140             pointLightCountMostrar++;
141     }
142     shaderList[0].SetPointLights(pointLightsMostrar,
            pointLightCountMostrar);
143
144
145
146     color = glm::vec3(1.0f, 1.0f, 1.0f);
147
148     model = glm::mat4(1.0);
149     model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f));
150     model = glm::scale(model, glm::vec3(30.0f, 1.0f, 30.0f));
151     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(
            model));
152     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
153
154     pisoTexture.UseTexture();
155     Material_opaco.UseMaterial(uniformSpecularIntensity,
            uniformShininess);
156     meshListModel[2]->RenderMeshModel();
157
158
159
160
161     //Rover: Sustituir por su propio Rover con jerarqu a
162     //rover importado
163     model = roverBase;
164     model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
165     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(
            model));
166     Rover_M.RenderModel();
167
168
169     ///================================================
170     ////
171     //ave importado
172     //
173     model = naveBase;
```

```
174                 model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
175                 model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f,
                        0.0f));
176                 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(
                        model));
177                 Nave_M.RenderModel();
178
179
180                 //fuego izq
181                 model = naveFuegoIzqBase;
182                 model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
183                 model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 0.0f,
                        1.0f));
184                 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(
                        model));
185                 Fuego_M.RenderModel();
186
187                 //fuego der
188                 model = naveFuegoDerBase;
189                 model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
190                 model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 0.0f,
                        1.0f));
191                 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(
                        model));
192                 Fuego_M.RenderModel();
193
194
195
196                 //Dado texturizado por codigo
197                 model = glm::mat4(1.0);
198                 model = glm::translate(model, glm::vec3(-5.0f, 5.0f, 6.0));
199                 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(
                        model));
200                 dadoTexture.UseTexture();
201                 meshListModel[4]->RenderMeshModel();
202
203                 //Holocron
204                 model = glm::mat4(1.0);
205                 model = glm::translate(model, glm::vec3(-7.0f, 7.0f, 6.0));
206                 model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
207                 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(
                        model));
208                 holocronTexture.UseTexture();
209                 meshListModel[5]->RenderMeshModel();
210
211
212                 //
213                 //
214                 //
215                 //
216                 //
217                 //
```

```
                //
                //
                //A adir modelo del avi n



                //Agave   qu   sucede si lo renderizan antes del helic ptero?
                model = glm::mat4(1.0);
                model = glm::translate(model, glm::vec3(0.0f, 1.0f, -4.0f));
                model = glm::scale(model, glm::vec3(4.0f, 4.0f, 4.0f));
                glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(
                    model));

                //blending: transparencia o traslucidez
                glEnable(GL_BLEND);
                glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
                AgaveTexture.UseTexture();
                Material_opaco.UseMaterial(uniformSpecularIntensity,
                    uniformShininess);
                meshListModel[3]->RenderMeshModel();
                glDisable(GL_BLEND);

                glUseProgram(0);

                mainWindow.swapBuffers();
        }

        return 0;
}
```

Conlusiones

Para la realización de esta practica se me complico un poco entender como hacer que las luces se apagaran al momento de presionar una tecla pero con lo que explico el profe fue un poco menos complicado ya que nos había dado una idea, la forma mas fácil que encontré para implementar eso fue iterar el arreglo de las luces puntuales y ahí hacer condicionales para cada una de las luces de los motores, considero que quizá no fue complicado de entender que hacer, pero si a la hora de hacerlo lleva tiempo ya que es fácil enredarse a la hora de instanciar las luces, acomodarlas, y hacer todo de las verificaciones.

Referencias

[1] Wolff, D. (2018). *OpenGL 4 Shading Language Cookbook: Build high-quality, real-time 3D graphics with OpenGL 4.6 using GLSL 4.6* (3rd ed.). Packt Publishing.

[2] Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). *OpenGL Programming Guide: The official guide to learning OpenGL, version 4.3* (8th ed.). Addison-Wesley Professional.

[3] Khronos Group. (2023). *OpenGL reference pages: glVertexAttribPointer*. `https://registry.khronos.org/OpenGL-Refpages/gl4/html/glVertexAttribPointer.xhtml`

[4] Khronos Group. (2023). *OpenGL reference pages: glDrawArrays.* https://registry.khronos.org/OpenGL-Refpages/gl4/html/glDrawArrays.xhtml

[5] Khronos Group. (2023). *OpenGL shading language specification, version 3.30.* https://registry.khronos.org/OpenGL/specs/gl/GLSLangSpec.3.30.pdf